Evidence-Grounded Criterion-Level Clinical-Trial Screening with a Fine-Tuned Small Transformer and From-Scratch Efficient Inference

Muhammad Ibrahim Malik (23i-0694)
Department of Computer Science
FAST-NUCES Islamabad, Pakistan
Generative AI, Assignment 2 — Instructor: Dr. Akhter Jamil

***Abstract*—Screening patients against the eligibility criteria of clinical trials is slow, manual work. We study whether a small, locally executable decoder Transformer can make criterion-level eligibility judgements and cite the patient-note sentences that support them. Using the TrialGPT Criterion Annotations dataset with a fixed patient-grouped split (37/8/8 patients), we fine-tune Qwen2.5-1.5B-Instruct with LoRA to emit a structured label and supporting sentence IDs. On the untouched test split (206 criteria) accuracy rises from 0.325 to 0.743, macro-F1 from 0.256 to 0.545 and evidence micro-F1 from 0.123 to 0.508. We re-implement the model's forward pass from its weights, with an explicit pre-allocated key/value cache, a tiled online-softmax attention kernel and greedy speculative decoding with a fine-tuned 0.5B draft model, and verify each against a reference path token-by-token. On a T4 GPU, KV caching gives up to 4.1× lower latency for 512-token prompts; tiled attention uses 13× less memory than explicit attention at 2048 tokens but runs 9× slower in pure PyTorch; speculative decoding is exact in all 96 runs but gains at most 1.07× on the GPU, against up to 1.27× on a CPU, where decoding is bandwidth bound. A batched decoder with per-row cache lengths screens a 21-criterion trial 4.2× faster on a laptop CPU. Everything is exposed in a browser application that produces auditable screening reports. The system is an educational prototype, not a clinical decision tool.**

***Index Terms*—clinical trial matching, eligibility screening, LoRA, KV cache, online softmax, speculative decoding, evidence extraction**

I. Introduction and Problem Statement

Clinical trials define who may enrol through free-text inclusion and exclusion criteria. Matching a patient record against dozens of criteria is labour-intensive, and recent work shows that large language models (LLMs) can assist by judging each criterion and explaining the decision [1], [2]. These systems rely on large proprietary models, which raises cost, privacy and reproducibility concerns for clinical text.

Problem. Given (i) a patient description split into numbered sentences, (ii) a criterion type (inclusion or exclusion) and (iii) the criterion text, predict the expert eligibility category, keeping the dataset's categories *not enough information* and *not applicable* rather than collapsing them into yes/no, and return the IDs of the patient sentences that support the decision. Absence of information must never be read as evidence of eligibility or ineligibility.

Contributions. (1) A LoRA-fine-tuned Qwen2.5-1.5B model that emits a structured label and sentence-ID evidence, evaluated before and after fine-tuning on a fixed, leakage-checked patient split. (2) A from-scratch Qwen2 forward pass with an explicit KV cache, including batched decoding of sequences with different cache lengths and prefix reuse across criteria. (3) A tiled online-softmax attention kernel. (4) Greedy speculative decoding with cache rollback. (5) An empirical analysis of when each mechanism helps on a T4 GPU and on a laptop CPU. (6) A browser application with auditable, downloadable reports.

II. Related Work

LLM-based patient–trial matching. Koopman and Zuccon [3] released patient topics (the `sigir-*` patients in our data) and trial relevance judgements. TrialGPT [1] uses GPT-4 to predict criterion-level eligibility with explanations and cited sentences, then aggregates and ranks trials; its expert-annotated criterion predictions form our dataset. Wornow et al. [2] show zero-shot GPT-4 matching on a cohort benchmark. Both depend on proprietary models; neither studies small open models under supervised adaptation, and their explanations are free text that is hard to verify. We instead require sentence IDs, which can be checked automatically, and use only a 1.5B-parameter open model.

Open decoder models and parameter-efficient fine-tuning. Qwen2/2.5 [4], [5] and Llama 3 [6] provide small instruction-tuned decoders built on the Transformer [7] with rotary position embeddings (RoPE) [8] and grouped-query attention (GQA) [9]. LoRA [10] trains low-rank updates to frozen weights, and QLoRA [11] additionally quantises the base model to 4 bits. LoRA's limitation is a smaller update space than full fine-tuning; QLoRA's is quantisation error and dependence on CUDA-only kernels. Because a 1.5B model fits a 16 GB T4 in fp16, we use plain LoRA and avoid quantisation error.

Efficient inference. Caching keys and values turns per-token decoding from re-processing the whole sequence into processing one token [12]; vLLM [13] manages cache memory in pages. Our setting is single-user, so we pre-allocate a contiguous cache and add rollback and ragged batching. Online softmax [14] computes softmax normalisers in one pass; Rabe and Staats [15] and FlashAttention [16] use it to tile attention so the full score matrix is never stored.

TABLE I
PATIENT-GROUPED SPLIT (EXPERT LABELS).

Split	Pat.	Ann.	inc	n.inc	exc	n.exc	NEI	N/A
Train	37	603	88	13	8	292	174	28
Val	8	206	35	3	2	101	53	12
Test	8	206	27	7	0	91	66	15

FlashAttention's speed comes from a fused, IO-aware CUDA kernel, which an educational PyTorch loop cannot match; we measure that gap. Speculative decoding [17], [18] lets a small draft model propose tokens that the target verifies in one pass, giving identical greedy output. The reported speed-ups assume the target pass is much more expensive than the draft pass; we test that assumption on two kinds of hardware.

III. MATERIALS AND METHODS

A. Dataset and Patient-Grouped Split

We use only `ncbi/TrialGPT-Criterion-Annotations`: 1015 criterion annotations over 53 patients and 103 trials. Each row has the patient note (already split into numbered sentences, 3–14 per note, about 100 words), criterion type and text, the expert label and the expert evidence sentences. The four GPT-4 columns (explanation, its correctness, sentences and label) are dropped when the split files are written and are never used. One row has an empty criterion text; it is kept as an empty string. 610 rows (60%) have no expert evidence sentences.

Following the protocol, unique patient IDs are sorted, shuffled with `numpy.random.default_rng(42)` and assigned 37/8/8 to train/validation/test. The script asserts that the three patient sets are disjoint and that every annotation appears in exactly one split, and writes a manifest with patient IDs and counts (Table I). The test split contains no *excluded* example, so that class cannot be scored on test. Prompts and checkpoints were chosen on validation only; the test split was evaluated once, after the model, prompt and decoding settings were frozen.

B. Model Selection

Table II compares the candidates. The deciding constraints were: training on a free 16 GB T4 without quantisation; CPU inference on a 16 GB-RAM laptop for the live demonstration; a smaller model with the *same tokenizer* as a speculative draft; and a permissive licence. Qwen2.5-1.5B-Instruct meets all four: 1.54B parameters (3.1 GB in fp16, 6.2 GB in fp32), Apache-2.0, a 32k context that easily covers our prompts (at most 650 tokens), and Qwen2.5-0.5B-Instruct shares its 151,936-entry vocabulary. Its GQA layout (12 query heads, 2 KV heads, head dimension 128, 28 layers) makes the KV cache small: $2 \times 28 \times 2 \times 128 \times 2\,\mathrm{B} = 28\,\mathrm{KiB}$ per token in fp16, six times less than with full multi-head attention. We rejected Llama-3.2-1B because it requires gated access under a custom licence and its only same-tokenizer sibling (3B) is too slow for CPU demonstrations. Qwen2.5-3B would need QLoRA on a T4 and runs at under 1 token/s on our laptop.

TABLE II
CANDIDATE OPEN-WEIGHT DECODERS (PUBLIC MODEL CARDS; MEMORY = WEIGHTS ONLY).

Model	Params	Layers	Q/KV heads	fp16	Licence	Role
Qwen2.5-1.5B-Instr.	1.54B	28	12/2	3.1 GB	Apache-2.0	**target**
Qwen2.5-0.5B-Instr.	0.49B	24	14/2	1.0 GB	Apache-2.0	**draft**
Qwen2.5-3B-Instr.	3.09B	36	16/2	6.2 GB	Qwen research	rejected
Llama-3.2-1B-Instr.	1.24B	16	32/8	2.5 GB	Llama 3.2 (gated)	rejected
Llama-3.2-3B-Instr.	3.21B	28	24/8	6.4 GB	Llama 3.2 (gated)	rejected

C. Input/Output Format and Tokenization

Each example becomes a chat prompt: a system message stating that only information in the note may be used and that absence of information is not evidence; then the numbered note, the criterion type, the criterion text, the four allowed labels for that type with one-line definitions, and the required JSON schema. The target is {`"label": "<label>", "evidence_sentence_ids": [IDs]`}, followed by the end-of-turn token. Outputs are parsed by extracting the first JSON object. A label outside the criterion type's label set becomes *invalid output* and counts as an error; it is never silently mapped to another class. Evidence IDs outside the note are discarded. Prompts tokenized with the Qwen chat template have 357/369/340 tokens on average (train/val/test) and at most 650. The training sequence limit is 1024 tokens, so **no example is truncated** (verified and logged). If truncation were needed, the script would trim the prompt from the left and never the answer, which could remove early note sentences. This is why it is logged.

D. Fine-Tuning

We apply LoRA (rank 16, $\alpha = 32$, dropout 0.05) to all attention and MLP projections (q, k, v, o, gate, up, down), which is 18.5M trainable parameters (1.18%). The objective is next-token cross-entropy on the answer tokens only; prompt tokens are masked with -100. We train for 3 epochs with AdamW, learning rate 2×10^{-4}, cosine schedule with 5% warm-up, micro-batch 2 with gradient accumulation 8 (effective batch 16, 114 steps), fp16 mixed precision, gradient checkpointing and seed 42. A first attempt with micro-batch 4 ran out of memory on the T4: with a 151,936-token vocabulary the fp32 logits alone take about 1.6 GB per batch. Validation loss is computed every 10 steps, and the best checkpoint (step 80, validation loss 0.113) is kept. Training took 15.8 minutes on one T4 (Fig. 1). The 0.5B draft is fine-tuned the same way (8.8M LoRA parameters, 8.0 minutes) so that its proposals follow the target's output format. The adapter and logs are published on Hugging Face.

E. System Architecture

Fig. 2 shows the pipeline. Only the weights come from Hugging Face. The forward pass, cache, attention kernels and decoding loops are our own code (the `src/` package of the repository).

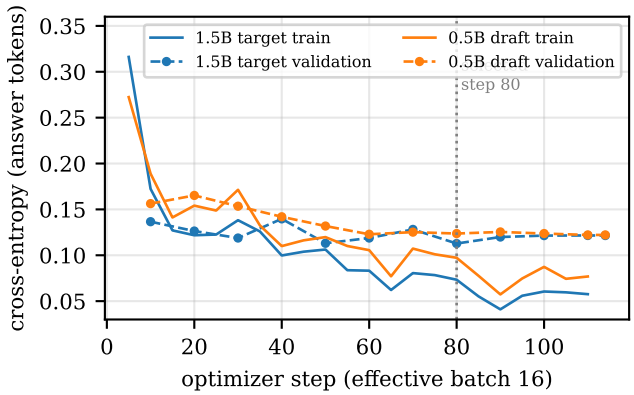


Fig. 1. Training and validation loss. Validation loss flattens after step 80 while training loss keeps falling (beginning of overfitting); the step-80 checkpoint is selected by validation loss.

Algorithm 1 Tiled causal attention with online softmax

for each query block Q_i (positions P_i) **do**
  $m \leftarrow -\infty,\ \ell \leftarrow 0,\ a \leftarrow 0$
  for each key block K_j, V_j with $\min(\text{pos}(K_j)) \le \max(P_i)$ **do**
    $S \leftarrow Q_i K_j^\top / \sqrt{d}$; mask S where key pos $>$ query pos
    $m' \leftarrow \max(m, \text{rowmax}(S))$; $c \leftarrow e^{m-m'}$; $E \leftarrow e^{S-m'}$
    $\ell \leftarrow c\,\ell + \text{rowsum}(E)$; $a \leftarrow c\,a + EV_j$; $m \leftarrow m'$
  end for
  $O_i \leftarrow a/\ell$
end for

Algorithm 2 Greedy speculative decoding with cache rollback

target prefill on prompt $\rightarrow$ first token; $s \leftarrow$ prompt + [first]
while not EOS and $|out| < N$ **do**
  draft: feed $s[n_{draft}:]$, then propose $d_1..d_k$ greedily
  $t_{1..k+1} \leftarrow$ arg max target$([s_{-1}, d_1..d_k])$ ▷ one pass
  $n \leftarrow$ largest n with $d_{1..n} = t_{1..n}$; $s \mathrel{+}= d_{1..n} \,\|\, t_{n+1}$
  target.rollback$(|s|-1)$; draft.rollback$(\min(n_{draft}, |s|-1))$
end while

F. Manual Forward Pass and KV Cache

For new tokens at absolute positions $p_0, \ldots, p_0 + T - 1$, each layer computes $x = \text{RMSNorm}(h)$, q, k, v projections (with biases), RoPE with $\theta = 10^6$ applied to q and k at their absolute positions, attention, an output projection and residual, then a SwiGLU MLP and residual. The cache stores *post-RoPE* keys and values in pre-allocated tensors $K, V \in \mathbb{R}^{L \times B \times H_{kv} \times N_{max} \times d}$ with a length counter n. A forward call writes new keys/values to slots $[n, n+T)$, attends over slots $[0, n+T)$ and advances n. Position handling therefore reduces to one rule: query i of the call sits at position $n + i$ and may attend to key j iff $j \le n + i$. The same rule covers prefill ($n = 0$), single-token decoding ($T = 1$) and multi-token speculative verification. Each of the 2 KV heads is shared by 6 query heads (GQA). Rolling back after rejected speculative tokens only lowers n; stale slots are never read and are later overwritten. The uncached reference path re-runs the entire sequence at every step without any state.

Prefix reuse and ragged batching. All criteria for one patient share the system prompt and note (238 tokens in our example). (i) Sequential prefix reuse keeps the previous cache, rolls it back to the longest common token prefix with the next prompt and prefills only the remainder. (ii) Batched decoding prefills the shared prefix once, copies it into every row of a batched cache, prefills all criterion suffixes together right-padded (causal masking guarantees that real tokens never see the padding after them), then decodes all rows together. Each row b keeps its own length n_b: its new key is scattered to slot n_b, RoPE uses position n_b, and a per-row mask allows keys $0..n_b$ only. On CPU a decoding step is memory-bandwidth bound, so B rows cost little more than one.

G. Reference and Tiled Online-Softmax Attention

The reference computes $S = QK^\top / \sqrt{d}$ in fp32, applies the causal (and optional per-row) mask with $-\infty$, subtracts the row maximum and normalises. The tiled kernel (Algorithm 1) processes query blocks and streams key/value blocks, keeping per query row a running maximum m, normaliser ℓ and unnormalised output a [14], [16]. Key blocks entirely in the causal future are skipped, and masking is applied only to blocks that straddle the diagonal. Rows with no visible key so far keep $m = -\infty$; we substitute 0 in the exponent to avoid $\infty - \infty$.

H. Greedy Speculative Decoding

Algorithm 2 keeps the invariant that the target cache holds every committed token except the newest. Each round, the draft catches up on committed tokens it has not cached and proposes k tokens greedily. The target scores $[x_{last}, d_1..d_k]$ in one pass, which gives its own greedy choice $t_1..t_{k+1}$ after each prefix. The longest prefix with $d_i = t_i$ is accepted, then t_{n+1} is appended (a correction, or a free bonus token when all k match). Both caches are rolled back to drop rejected tokens. By construction the output equals greedy decoding of the target; we verify this rather than assume it.

I. Application Workflow

The Gradio application (started by double-clicking `run_app.bat`; it works offline once the models are cached) accepts a free-text note, which is split into numbered sentences automatically. Inclusion and exclusion criteria can be typed, loaded from a validation example, or fetched from the public ClinicalTrials.gov API by NCT ID. For each criterion it shows the label, cited sentence IDs and their text, and an explanation built *only* from the cited sentences. A *not enough information* result is explicitly reported as missing information rather than as evidence. The overall outcome is an explicit rule, not a model output, because the dataset has no trial-level ground truth: *likely ineligible* if any inclusion criterion is *not included* or any exclusion criterion is *excluded*; otherwise *needs review* if any criterion is undecidable or invalid, or if more than half of the criteria are *not applicable*; otherwise *potentially eligible*. *Not applicable* criteria are otherwise ignored, and the number set aside is always shown. We added the majority-N/A rule after observing the base model in the application: it labels most criteria N/A, and ignoring them

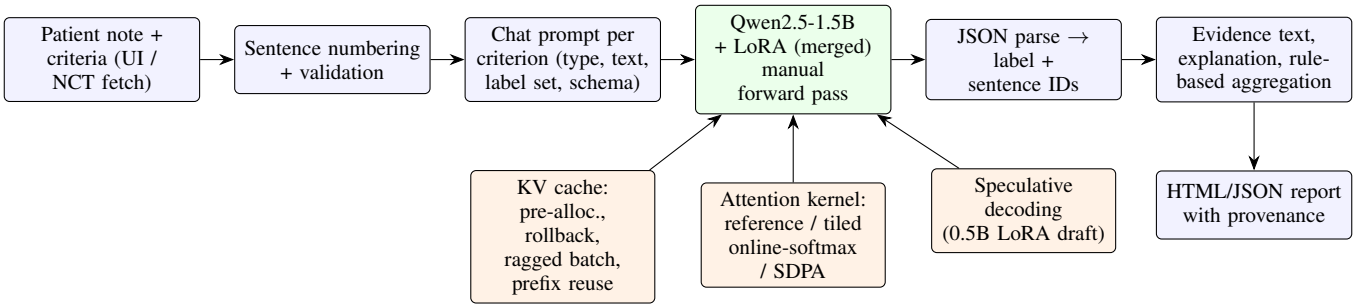


Fig. 2. Screening pipeline, from patient note and trial criteria to a criterion-level report. Orange modules are implemented from scratch and can be switched at run time in the application.

turned a 9/21-correct screening into a misleading *potentially eligible*. Input validation rejects empty notes, notes over 1200 tokens, more than 40 criteria or over-long criteria with a message instead of crashing. Reports are downloadable as HTML and JSON with provenance: UTC time, code commit, model and adapter IDs, decoding and attention settings, an input SHA-256 fingerprint and runtime. Every page carries the notice that this is an educational screening prototype. Users can switch live between the fine-tuned and base model (the LoRA update is unmerged and disabled in place), any decoding mode and any attention kernel. An inference lab runs the same prompt through each decoder and reports token identity, prefill/decode time, throughput, acceptance rate and cache size.

IV. Experimental Setup and Metrics

Hardware. Training and GPU benchmarks: Kaggle, one NVIDIA T4 (16 GB), PyTorch 2.11, fp16. Application and CPU benchmarks: laptop with an Intel Core i7-1255U (2 performance + 8 efficiency cores), 16 GB RAM, fp32.

Decoding. All evaluations use greedy decoding with at most 64 new tokens. Qwen's default generation configuration silently applies a repetition penalty of 1.1 even when sampling is off. We found this when our cached decoder disagreed with `generate()` while matching the model's own forward pass. We therefore override it to 1.0 everywhere, so all reported numbers are pure greedy decoding and directly comparable with our decoders.

Classification metrics. Accuracy; macro-F1 over the classes that occur in the gold labels of the evaluated subset (so the absent *excluded* class does not count as an automatic zero on test); per-class precision/recall/F1; and confusion matrices, overall and separately for inclusion and exclusion criteria.

Evidence metrics. Micro precision/recall/F1 pool TP/FP/FN of sentence IDs over all records. Macro scores average per-record P/R/F1 with explicit conventions for empty sets: gold and prediction both empty $\Rightarrow$ P=R=F1=1 (correctly cited nothing); gold empty, prediction non-empty $\Rightarrow$ P=0, R=1, F1=0; gold non-empty, prediction empty $\Rightarrow$ P=1, R=0, F1=0. Because 60% of records have no gold evidence, the macro average is inflated by empty-empty agreement, so we also report it restricted to records with non-empty gold evidence. We regard micro-F1 and this restricted score as the honest measures.

TABLE III
Criterion-level results, base vs. LoRA (greedy, same prompt). Ev. = evidence; ne = gold non-empty.

Split	Model / subset	Acc	Macro-F1	Ev. μF1	Ev. MF1	Ev. F1$_{ne}$
Val	Base, all	0.345	0.221	0.124	0.336	0.168
	LoRA, all	**0.733**	**0.431**	**0.522**	**0.729**	**0.535**
Test	Base, all	0.325	0.256	0.123	0.484	0.111
	Base, inclusion	0.160	0.149	0.235	0.449	0.081
	Base, exclusion	0.420	0.291	0.088	0.505	0.195
	LoRA, all	**0.743**	**0.545**	**0.508**	**0.813**	**0.533**
	LoRA, inclusion	0.787	0.424	0.594	0.670	0.671
	LoRA, exclusion	0.718	0.433	0.158	0.895	0.143

Benchmark protocol. Every timed configuration gets one untimed warm-up run, then three timed runs. On GPU the timer calls `torch.cuda.synchronize()` before and after, and we report the median. Peak memory is the CUDA allocator peak above the memory already in use (weights excluded). KV benchmarks use fixed output lengths (EOS ignored) on synthetic clinical prompts of 64/256/512 tokens. Speculative and attention end-to-end benchmarks use 10 validation prompts plus 2 free-text prompts, so the test split is never used for tuning.

V. Results and Discussion

A. Before vs. After Fine-Tuning

Table III compares the same model, prompt and greedy decoding before and after fine-tuning on identical examples. Fine-tuning more than doubles accuracy on both splits, and the validation and test results agree closely (0.733 vs. 0.743), so no tuning overfitted validation. The base model's main failure is predicting *not applicable* for most criteria (101 of 206 validation predictions), and it rarely cites the right sentences (evidence micro precision 0.079 on test). After fine-tuning, invalid outputs drop to zero, and the model reproduces the output format and the dataset's label conventions.

B. Per-Class Behaviour

On test (Fig. 3), the fine-tuned model is strong on *included* (P 0.92, R 0.82, F1 0.86) and *not excluded* (P 0.71, R 0.99,

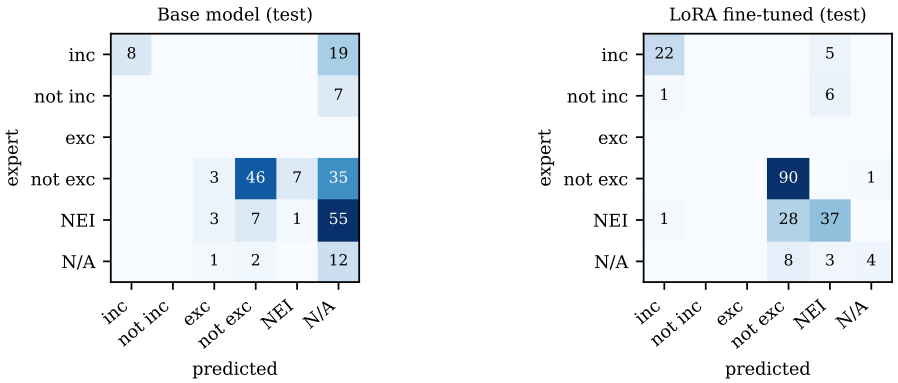


Fig. 3. Test confusion matrices (rows: expert, columns: predicted; NEI = not enough information, N/A = not applicable). The base model defaults to N/A; the fine-tuned model's dominant error is NEI→not excluded.

F1 0.83), moderate on *not enough information* (F1 0.63) and weak on *not applicable* (F1 0.40, recall 0.27). It never predicts *not included* (0/7). With only 13 training examples, and 8 for *excluded*, these minority classes are not learned. That is the main reason macro-F1 (0.545) lags accuracy. Class re-weighting or oversampling is the obvious next step, but we did not tune it, to keep the test protocol clean within the time budget.

C. Evidence Localisation

Evidence quality differs sharply by criterion type. For inclusion criteria, micro-F1 is 0.594 (recall 0.74). For exclusion criteria, recall is only 0.094: the model usually answers *not excluded* without citing anything, while experts often cite the sentence that rules the exclusion out (e.g. the age sentence for "less than 18 years"). The empty-set convention matters a great deal. Exclusion macro-F1 is 0.895 because most exclusion records have empty gold *and* empty predicted evidence, whereas F1 on records with gold evidence is 0.143. We therefore do not treat the macro score as evidence of good localisation.

D. Error Analysis

A rule-based taxonomy (`scripts/error_analysis.py`) assigns every test prediction to the first matching category (Table IV). Representative cases, identified by annotation ID:

1) **Missing information read as negative** (#169, #174, 28 cases): "eGFR <30" and "previous axillary surgery" are not mentioned; experts say *not enough information*, the model says *not excluded*. This is the most frequent error and exactly the failure the task warns about: absence treated as a negative finding. It mirrors a training-label asymmetry (292 *not excluded* vs. 174 NEI).
2) **Contradictory evidence** (#161, #162, 6 cases): for "patients undergoing elective carotid endarterectomy" the note describes a cardiac catheterisation. Experts say *not included*; the model answers *not enough information* while citing sentences 0–2, which actually decide the criterion.
3) **Under-confident** (#176): "no history of central venous catheterisation" for a patient who had femoral *arterial* catheterisation. Experts say *included* citing sentence 0; the model says NEI. Distinguishing arterial from venous access requires domain knowledge.
4) **Missed not-applicable** (#422, 11 cases): "if female, using birth control" for a girl with no sexual partners. Experts say *not applicable* (sentence 3); the model says NEI.
5) **Label ambiguity** (#322): a pregnancy/contraception exclusion for a 48-year-old man. The model predicts *not applicable*; experts chose *not excluded*. Both are defensible, which suggests that part of the N/A error is annotation ambiguity rather than model failure.
6) **Over-commitment** (#678): mild sleep-disordered breathing requires snoring on ≥3 nights/week for ≥3 months. The note says "nighttime snoring" without frequency or duration. Experts say NEI; the model says *included* (same evidence sentence).
7) **Opposite decision** (#164): "surgical lower-limb revascularisation" is predicted *included* from a groin pseudoaneurysm after catheterisation, citing sentences 0 and 2. The cited text is relevant but does not support the conclusion: an unsupported inference.
8) **Correct label, unsupported** (#196, 13 cases): "less than 18 years of age" for a 40-year-old is correctly *not excluded*, but no sentence is cited although sentence 0 states the age.
9) **Correct label, over-citation** (#307, 15 cases): a long, complex inclusion criterion is correctly NEI, but the model cites all five sentences where experts cite one, lowering evidence precision.

In the application, every one of these cases is traceable: each row links its label to the cited sentence text, and a NEI result lists the criterion under "missing information" instead of counting for or against the patient.

TABLE IV
ERROR TAXONOMY ON THE TEST SPLIT (FINE-TUNED MODEL, 206 CRITERIA; FIRST MATCHING RULE).

Category	Count
Missing information read as negative (NEI→not exc./inc.)	28
Missed *not applicable*	11
Cited evidence yet answered NEI (contradictory)	6
Answered NEI although note decides the criterion	5
Decided a criterion experts judged undecidable	1
Opposite decision / other label error	2
Correct label, no evidence where experts cited some	13
Correct label, cited sentences experts did not	15
Correct label, missed some expert evidence	2

TABLE V
KV CACHE VS. NO CACHE ON A T4 (FP16, MEDIAN OF 3). MEM = PEAK MEMORY ABOVE WEIGHTS (MIB).

Prompt	New	Equal	No-cache s	KV s	Speed-up	Mem_{nc}	Mem_{kv}
64	16	✓	0.70	0.68	1.02	6.7	9.9
64	64	@20	2.80	2.71	1.03	11.2	9.9
64	128	@20	5.79	5.37	1.08	14.7	12.3
256	16	✓	1.26	0.67	1.88	18.8	25.3
256	64	✓	5.40	2.69	2.01	22.1	26.5
256	128	✓	11.10	5.33	2.08	30.9	28.3
512	16	✓	2.51	0.75	3.36	48.9	61.2
512	64	✓	10.58	2.73	3.88	56.2	62.5
512	128	✓	22.29	5.39	4.14	68.3	64.3

E. KV-Cached Decoding

Correctness. Our forward pass matches Hugging Face logits to within 3×10^{-5} (fp32) with every attention kernel, both uncached and when the input is fed in arbitrary chunks through the cache. Under greedy decoding, cached, uncached and `generate()` outputs are token-identical in unit tests at three prompt/output lengths (fp32). This includes a 64-token-prompt case, prefix reuse, and ragged batching with all three kernels.

Mismatch investigation. On the T4 in fp16, 7 of 9 configurations are token-identical. The two exceptions (prompt 64, 64 and 128 new tokens) both diverge at token 20. Re-running that prompt in fp32 shows cached and uncached paths agree on all tokens, and at step 20 the top two candidates (" and" vs. ",") differ by only 0.0003 in a logit of about 26. fp16 has a resolution of about 0.016 at that magnitude, so the different summation orders of the two paths (one query over a cache vs. a full causal pass) flip the argmax. The mismatch is a numerical near-tie, not a cache bug.

Performance. Table V and Fig. 4 show that the benefit grows with context. Without a cache, decode throughput falls from 22.9 to 5.7 tokens/s as the prompt grows from 64 to 512 tokens, because every step re-processes the whole sequence. With the cache it stays at about 24 tokens/s, and latency improves 3.4–4.1× at 512 tokens. At 64 tokens the gain is only 2–8%: a batch-1 step of a 1.5B model on a T4 is dominated by kernel-launch and Python overhead (28 layers × many small kernels), not arithmetic, so recomputing 64 extra tokens costs almost nothing. Peak extra memory is slightly *higher* with the cache for short outputs, because the cache is pre-allocated for the full prompt+output length, and lower for long ones, where the uncached path's activations over the full sequence dominate. Analytically, the cache grows by 28 KiB per token in fp16.

F. Tiled Online-Softmax Attention

Correctness. 29 unit-test cases (prefill, single-token decode with offset, multi-token verification, ragged block sizes 7×13 and 16×16, non-causal, Tq>block) agree with the reference within 10^{-5}. Scores of order 10^4 remain finite, and changing future keys does not change earlier outputs. Against an fp64 ground truth (T=256, fp32 inputs) the maximum absolute errors are 9.8×10^{-7} (reference), 1.9×10^{-6} (tiled) and 1.1×10^{-6} (SDPA). On the T4 with fp16 inputs, errors against an fp32-accumulated reference are $\approx 9.6 \times 10^{-4}$ for all kernels, dominated by rounding the output to fp16. Inside the full model, tiled and SDPA attention give token-identical outputs to the reference on all 10 validation prompts.

Performance. Fig. 5 shows the expected trade-off. At T=2048 (causal) the reference needs 576 MiB of extra memory, because the $12 \times T^2$ fp32 score, exponent and probability matrices are materialised; tiling needs 43 MiB (13× less) and grows linearly. However, tiling is 9× slower (174 vs. 20 ms). Each 64×64 tile is a separate Python-level iteration that launches several small kernels and re-reads Q, m, ℓ and a from GPU memory. FlashAttention's advantage comes from fusing that loop into one kernel that keeps the tile statistics in on-chip SRAM. PyTorch's fused SDPA (2.6 ms, 18.5 MiB) shows what fusion buys. Our implementation demonstrates the algorithm and its memory behaviour, not the IO-aware kernel.

G. Speculative Decoding

The output of speculative decoding was token-identical to target-only greedy decoding in **all 72 GPU runs**, and in unit tests with a deliberately perturbed draft that forces rejections at $k \in \{1, 3, 5\}$. Table VI and Fig. 6 summarise the T4 results. Fine-tuning the draft is decisive for this task: on criterion prompts the LoRA draft's proposals are accepted 97–98% of the time, against 70–80% for the base draft. The target then advances 3.8–4.0 tokens per pass at $k \geq 4$, yet wall-clock speed-up is only 1.03–1.07×. With the base draft, or on free text (acceptance 0.49–0.72), speculative decoding is 11–36% *slower*. Its peak extra memory is 7 MiB higher (two caches), plus 1 GB for the draft weights.

The reason is the cost ratio. Speculative decoding pays off when a draft step is much cheaper than a target step. On a T4 at batch 1 both models are launch-bound: the 0.5B draft has 24 layers against the target's 28, so a draft step costs about 80% of a target step even though it has a third of the parameters. On the laptop CPU, decoding is memory-bandwidth bound and step cost scales with the bytes of weights read (2 GB vs. 6.2 GB in fp32). The same algorithm then gives a 1.17× speed-up on criterion prompts with the LoRA draft at $k = 4$, and 1.23–1.27× on free text even with only 59–73% acceptance,

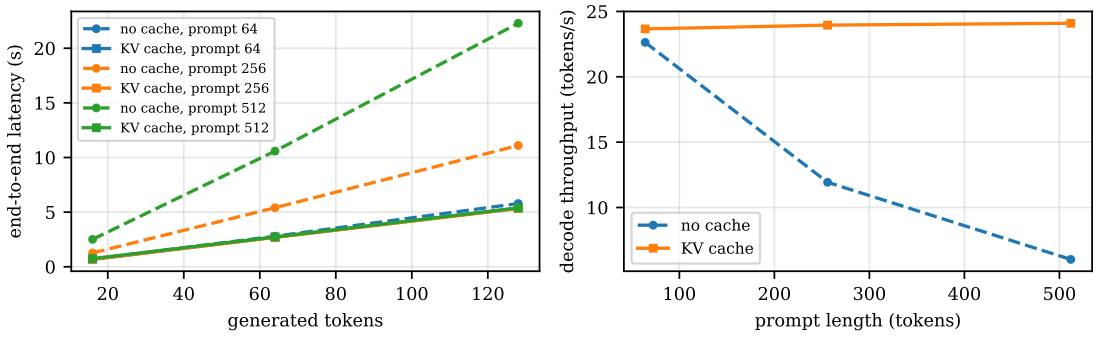


Fig. 4. Left: latency vs. generated tokens. Right: decode throughput vs. prompt length (T4, fp16). Uncached throughput collapses with context; cached throughput is flat.

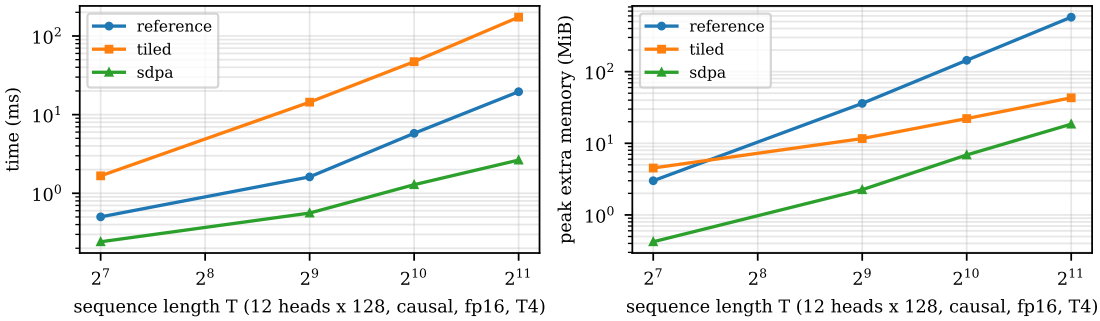


Fig. 5. Attention kernels on a T4 (causal, fp16, 12 heads × 128). Tiled memory grows linearly instead of quadratically, but the unfused loop is slower than both the reference and fused SDPA.

TABLE VI
SPECULATIVE DECODING (MEAN OVER PROMPTS; ACCEPTANCE = ACCEPTED / DRAFTED). T4 FP16 MEDIAN OF 3; CPU (I7-1255U, FP32) MEDIAN OF 2.

Prompts	Draft	k	Accept	Tok/pass	Speed-up	Equal
Criterion (10)	base 0.5B	2	0.80	2.20	0.89	10/10
	base 0.5B	4	0.70	2.83	0.81	10/10
	LoRA 0.5B	2	0.97	2.59	1.04	10/10
	LoRA 0.5B	4	0.97	3.82	**1.07**	10/10
Free text (2)	base 0.5B	4	0.58	3.24	0.75	2/2
	LoRA 0.5B	4	0.61	3.33	**1.27**	2/2
CPU, criterion (4)	base 0.5B	4	0.61	2.61	0.98	4/4
	LoRA 0.5B	4	1.00	3.85	**1.17**	4/4
CPU, free text (2)	base 0.5B	4	0.59	3.28	1.24	2/2
	LoRA 0.5B	4	0.61	3.33	**1.27**	2/2

where the T4 was 12–36% slower (Table VI). Outputs were identical in all 24 CPU runs. The speed-up is a property of the hardware regime, not only of the acceptance rate.

H. Batched Screening on CPU

We screened one validation patient against a 21-criterion trial (mean prompt 357 tokens, of which 238 are shared) on the laptop CPU, one run per method because each takes minutes. Sequential KV decoding took 255 s. Sequential decoding with prefix reuse took 198 s (1.29×): the shared note is prefilled once instead of 21 times. Batched decoding took **61 s (4.2×)**: 43.5 s of prefill and 17.4 s for 16 decoding steps. One step for all 21 rows costs about 1.1 s, against 0.5 s for a single row, because a CPU decoding step mostly streams the 6.2 GB of fp32 weights from memory and that cost is shared across rows. All three methods produced identical outputs for all 21 criteria. We also tried bf16 weights (prefill 5× slower: this CPU lacks native bf16 matrix units) and PyTorch dynamic int8 quantisation (slower, and 0/8 outputs matched fp32), and rejected both.

I. Application

Figs. 7 and 8 show the screening view and the inference lab. On the laptop, the application loads the merged model in about 15 s, screens a 10-criterion trial in about 33 s with batched decoding, and reproduces the evaluation script's predictions exactly (CPU fp32 vs. GPU fp16) on the checked examples. In a single lab run on one 369-token criterion prompt, uncached decoding took 76.8 s, cached 13.1 s, `generate()` 10.5 s and speculative decoding with the LoRA draft 8.2 s, all producing identical tokens.

VI. LIMITATIONS

(1) Small data: 603 training annotations from 37 patients, with minority classes of 8–13 examples. The test split has no *excluded* cases, so that class is untested and the 8-patient test

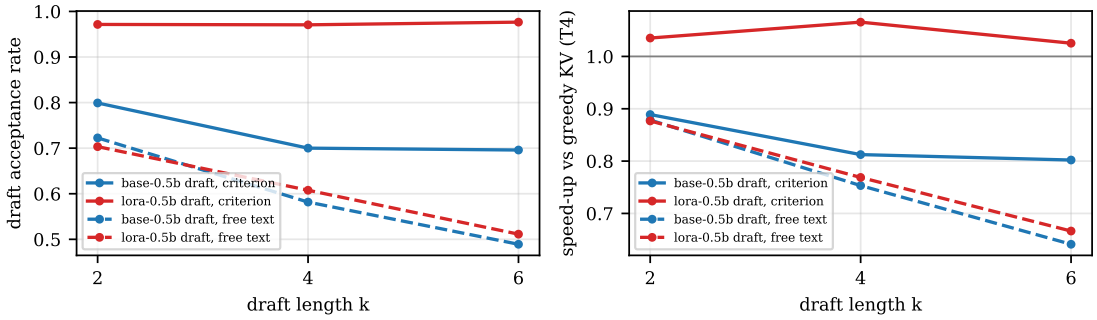
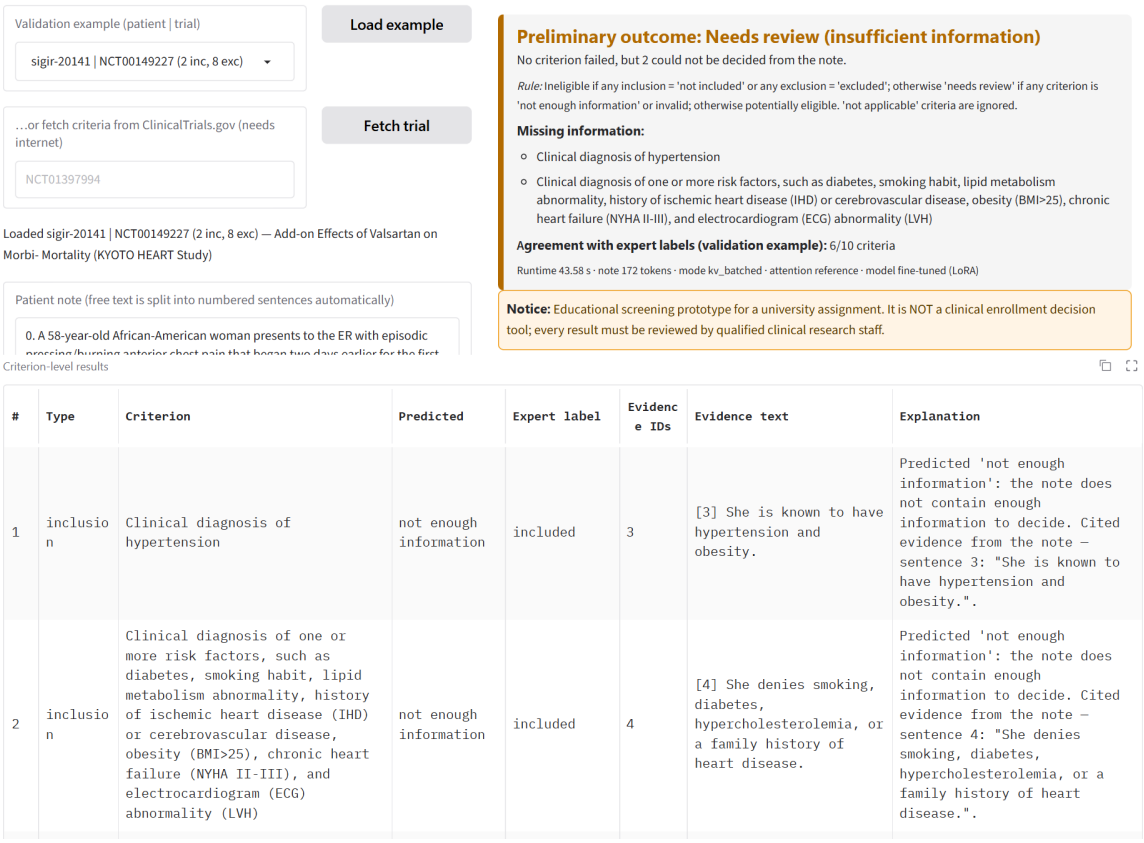


Fig. 6. Speculative decoding on a T4: acceptance rate and speed-up vs. draft length k. Only the fine-tuned draft on in-domain prompts beats greedy decoding.



#	Type	Criterion	Predicted	Expert label	Evidence IDs	Evidence text	Explanation
1	inclusion	Clinical diagnosis of hypertension	not enough information	included	3	[3] She is known to have hypertension and obesity.	Predicted 'not enough information': the note does not contain enough information to decide. Cited evidence from the note — sentence 3: "She is known to have hypertension and obesity.".
2	inclusion	Clinical diagnosis of one or more risk factors, such as diabetes, smoking habit, lipid metabolism abnormality, history of ischemic heart disease (IHD) or cerebrovascular disease, obesity (BMI>25), chronic heart failure (NYHA II-III), and electrocardiogram (ECG) abnormality (LVH)	not enough information	included	4	[4] She denies smoking, diabetes, hypercholesterolemia, or a family history of heart disease.	Predicted 'not enough information': the note does not contain enough information to decide. Cited evidence from the note — sentence 4: "She denies smoking, diabetes, hypercholesterolemia, or a family history of heart disease.".

Fig. 7. Screening view (validation patient, 10 criteria): rule-based preliminary outcome, missing information, agreement with expert labels and run provenance (top); per-criterion predicted label, expert label, cited sentence IDs, evidence text and an explanation grounded in the cited sentences (bottom, first three rows). HTML and JSON reports are downloadable.

set gives wide confidence intervals. (2) The model has not learned *not included* and under-detects *not applicable*; it also tends to read missing information as *not excluded*, which in a real screening tool would wrongly keep ineligible patients in the pool. (3) Exclusion evidence recall is low (0.09). (4) Explanations are templated from the cited sentences. This is faithful by construction but not a reasoning trace, because the dataset's only free-text explanations are GPT-4 outputs, which we may not use. (5) Patient notes are short synthetic case descriptions, unlike real EHR text. (6) The tiled kernel is unfused and slow, and our decoders run at batch size 1 apart from per-patient batching. (7) The patient–trial outcome is a rule-based aggregation, not a validated eligibility decision. It is only as reliable as the criterion labels beneath it; ignoring *not applicable* criteria in particular can hide unchecked criteria, which is why the application reports how many were set aside and refuses a positive outcome when they are the majority. **The system is an educational screening prototype and must not be used for clinical enrollment.**

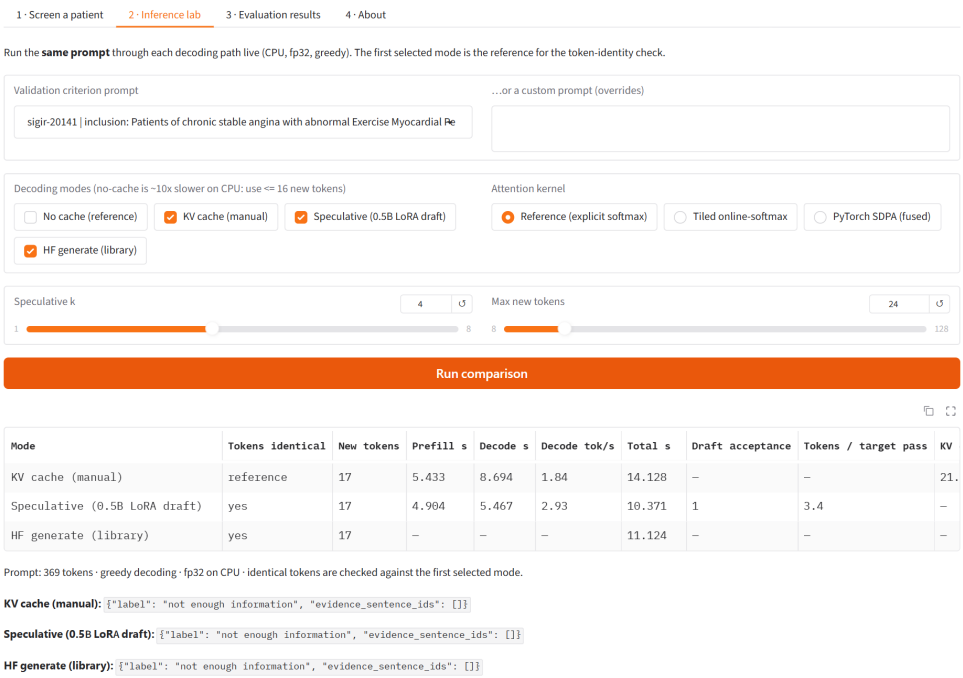


Fig. 8. Inference lab: the same 369-token criterion prompt decoded live on the laptop CPU with the manual KV cache (reference), speculative decoding with the LoRA draft and Hugging Face `generate()`. Token identity, prefill/decode time, throughput, draft acceptance and tokens per target pass are reported for each.

VII. Conclusion

A 1.5B open model with LoRA, trained in 16 minutes on a free GPU, reaches 0.743 accuracy and 0.545 macro-F1 on held-out patients and cites the right note sentences about half the time (evidence micro-F1 0.508), up from 0.325/0.256/0.123 before fine-tuning. Re-implementing inference showed that each efficiency mechanism helps only in the regime it targets. KV caching helps once context is long enough to dominate launch overhead. Tiling saves memory but needs kernel fusion to save time. Speculative decoding needs an in-domain draft *and* hardware where the draft is genuinely cheaper. Batching rows with different cache lengths was the largest practical win for interactive CPU screening. Future work: class-balanced training for the minority labels, an explicit penalty for citing nothing on exclusion criteria, and evaluation on real clinical notes.

References

[1] Q. Jin, Z. Wang, C. S. Floudas, F. Chen, C. Gong, D. Bracken-Clarke, E. Xue, Y. Yang, J. Sun, and Z. Lu, "Matching patients to clinical trials with large language models," *Nature Communications*, vol. 15, p. 9074, 2024. [Online]. Available: https://www.nature.com/articles/s41467-024-53081-z

[2] M. Wornow, A. Lozano, D. Dash, J. Jindal, K. W. Mahaffey, and N. H. Shah, "Zero-shot clinical trial patient matching with LLMs," *NEJM AI*, vol. 2, no. 1, 2025. [Online]. Available: https://arxiv.org/abs/2402.05125

[3] B. Koopman and G. Zuccon, "A test collection for matching patients to clinical trials," in *Proc. 39th Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2016, pp. 669–672. [Online]. Available: https://bevankoopman.github.io/papers/sigir2016_clinicaltrials_collection.pdf

[4] A. Yang *et al.*, "Qwen2 technical report," *arXiv preprint arXiv:2407.10671*, 2024. [Online]. Available: https://arxiv.org/abs/2407.10671

[5] Qwen Team, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2024. [Online]. Available: https://arxiv.org/abs/2412.15115

[6] A. Dubey *et al.*, "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024. [Online]. Available: https://arxiv.org/abs/2407.21783

[7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017. [Online]. Available: https://arxiv.org/abs/1706.03762

[8] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, p. 127063, 2024. [Online]. Available: https://arxiv.org/abs/2104.09864

[9] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 4895–4901. [Online]. Available: https://arxiv.org/abs/2305.13245

[10] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Int. Conf. Learning Representations (ICLR)*, 2022. [Online]. Available: https://arxiv.org/abs/2106.09685

[11] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023. [Online]. Available: https://arxiv.org/abs/2305.14314

[12] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, J. Heek, K. Xiao, S. Agrawal, and J. Dean, "Efficiently scaling transformer inference," in *Proc. Machine Learning and Systems (MLSys)*, vol. 5, 2023. [Online]. Available: https://arxiv.org/abs/2211.05102

[13] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proc. 29th Symp. Operating Systems Principles (SOSP)*, 2023, pp. 611–626. [Online]. Available: https://arxiv.org/abs/2309.06180

[14] M. Milakov and N. Gimelshein, "Online normalizer calculation for softmax," *arXiv preprint arXiv:1805.02867*, 2018. [Online]. Available: https://arxiv.org/abs/1805.02867

[15] M. N. Rabe and C. Staats, "Self-attention does not need $o(n^2)$ memory," *arXiv preprint arXiv:2112.05682*, 2021. [Online]. Available: https://arxiv.org/abs/2112.05682

[16] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022. [Online]. Available: https://arxiv.org/abs/2205.14135

[17] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proc. 40th Int. Conf. Machine Learning (ICML)*, 2023, pp. 19 274–19 286. [Online]. Available: https://arxiv.org/abs/2211.17192

[18] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," *arXiv preprint arXiv:2302.01318*, 2023. [Online]. Available: https://arxiv.org/abs/2302.01318

Appendix A
Use of AI Assistants

The implementation, experiments and this report were produced with the help of an AI coding assistant (Anthropic Claude, via Claude Code), which wrote most of the code and draft text under the author's direction. The author chose the overall approach, ran all training and benchmark jobs on Kaggle, checked results against the assignment rubric, and is responsible for every claim. All AI-written code was tested (40 unit tests comparing the manual model, cache, attention kernels and speculative decoder with reference implementations), and all numbers in this report come from logged result files in the repository. Several issues were found and fixed through these checks: a hidden repetition penalty in the default generation configuration, a GPU out-of-memory configuration, library-version incompatibilities on Kaggle, and an attention "fp64 ground truth" that was silently computed in fp32. Hugging Face Transformers, PEFT, PyTorch and Gradio are used as libraries. TrialGPT code was not used.

Appendix B
Reproducibility

- Code: https://github.com/ibrahimmlk/trialsieve
- Target adapter: https://huggingface.co/ibrahimmlk/trialsieve-qwen2.5-1.5b-lora
- Draft adapter: https://huggingface.co/ibrahimmlk/trialsieve-qwen2.5-0.5b-draft-lora
- Split and manifest: `python scripts/make_split.py` → `data/splits/split_manifest.json`
- Training and GPU evaluation: `notebooks/kaggle_finetune.ipynb`, `notebooks/kaggle_benchmarks_final.ipynb`
- Tests: `pytest tests`; figures: `python scripts/make_figures.py`; error analysis: `python scripts/error_analysis.py`
- Application: double-click `run_app.bat` (opens http://127.0.0.1:7860)